

Core Fundamentals Prep Rupam Pal

Interview Prep — Rupam Pal

Object-Oriented Programming

Q1. The four pillars — define each with a one-line code example.

```
// 1. Encapsulation: hide internal state, expose controlled access
public class Account
{
    private decimal _balance; // hidden
    public decimal Balance => _balance; // read-only exposure
    public void Deposit(decimal amount) { if (amount > 0) _balance += amount; }
}

// 2. Abstraction: expose *what*, hide *how*
public interface IPaymentProcessor { Task<bool> ChargeAsync(decimal amount); }
// Caller doesn't care if it's Stripe, Razorpay, or a bank API underneath.

// 3. Inheritance: reuse + specialize
public class Employee { public string Name { get; set; } public virtual decimal CalculatePay() => 0; }
public class SalariedEmployee : Employee { public decimal Salary { get; set; } public override decimal CalculatePay() => Salary; }

// 4. Polymorphism: same call, different behavior at runtime
Employee e = new SalariedEmployee { Salary = 50000 };
e.CalculatePay(); // resolves to SalariedEmployee's override at runtime (virtual dispatch)
```

Q2. Compile-time (overloading) vs runtime (overriding) polymorphism — a common bug.

```
public class Base { public virtual void Print() => Console.WriteLine("Base"); }
public class Derived : Base {
    public new void Print() => Console.WriteLine("Derived"); // 'new' hides, doesn't override!
}

Base b = new Derived();
b.Print(); // prints "Base" -- 'new' breaks virtual dispatch because the static
           // (compile-time) type Base is used, not the runtime type.
// Fix: use 'override' on a 'virtual' base method if polymorphic dispatch is intended.
```

Q3. Composition over inheritance — why, with an example?

Deep inheritance chains get fragile (changing a base class can break distant subclasses unexpectedly). Composition builds behavior by injecting smaller collaborators instead of extending a rigid hierarchy.

```
public interface IFlyBehavior { void Fly(); }
public class Duck
{
    private readonly IFlyBehavior _flyBehavior;
    public Duck(IFlyBehavior f) => _flyBehavior = f;
    public void PerformFly() => _flyBehavior.Fly();
}
public class Penguin { /* no IFlyBehavior injected -- no awkward override-to-throw */ }
```

Q4. Interface vs abstract class — when do you pick one over the other?

Abstract class: shared state plus some default implementation, “is-a” relationship, single inheritance only.

Interface: pure contract, multiple interfaces can be implemented, “can-do” capability. Rule of thumb: unrelated classes sharing a capability (`IDisposable`, `IComparable`) -> interface. A family of related types sharing real code -> abstract class.

Q5. Liskov Substitution Principle violation — the classic Rectangle/Square example.

```
// BUG: Square "is-a" Rectangle mathematically, but violates LSP as a class hierarchy
public class Rectangle { public virtual int Width { get; set; } public virtual int Height { get; set; }
public class Square : Rectangle
{
    public override int Width { set { base.Width = base.Height = value; } }
    public override int Height { set { base.Width = base.Height = value; } }
}
// Code that does rect.Width = 5; rect.Height = 10; expecting Area == 50
// silently breaks when 'rect' is actually a Square (Area becomes 100).

// FIX: don't force an inheritance relationship where behavior contracts diverge --
// use a shared IShape interface with independent implementations instead.
```

JavaScript Fundamentals

Q6. `var` vs `let` vs `const` — the classic loop closure bug.

```
// BUG: var is function-scoped, not block-scoped -- all 3 callbacks share the SAME i
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // prints 3, 3, 3
}

// FIX: let is block-scoped -- a NEW binding of i is created per iteration
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // prints 0, 1, 2
}
```

Q7. `==` vs `===` — a bug caused by type coercion.

```
console.log(0 == '');           // true (coercion: '' -> 0)
console.log(0 === '');          // false (no coercion, different types)
console.log(null == undefined); // true
console.log(null === undefined); // false
// Rule: always use === / !== unless you have a specific, documented reason for coercion.
```

Q8. `this` binding — the 4 rules, with a common bug.

```
const obj = {
  name: 'Order',
  items: [1, 2, 3],
  printItems() {
    // BUG: regular function loses 'this' -- inside forEach callback, 'this' is
    // undefined (strict mode) or global
    this.items.forEach(function (item) {
      console.log(this.name, item); // TypeError: Cannot read properties of undefined
    });
  }
};

// FIX 1: arrow function inherits 'this' lexically from enclosing scope
printItemsFixed() {
  this.items.forEach((item) => console.log(this.name, item));
}

// FIX 2: bind explicitly
this.items.forEach(function (item) { console.log(this.name, item); }.bind(this));
```

Precedence, highest to lowest: new binding > explicit (`call` / `apply` / `bind`) > implicit (`obj.method()`) > default (global/undefined in strict mode). Arrow functions ignore all 4 – they capture `this` from where they're *defined*, not called.

Q9. Hoisting — what gets hoisted, and the Temporal Dead Zone bug.

```
console.log(a); // undefined (var is hoisted AND initialized to undefined)
var a = 5;

console.log(b); // ReferenceError: Cannot access 'b' before initialization (TDZ)
let b = 10;
// let/const ARE hoisted to the top of the block, but stay uninitialized until
// their declaration line executes -- that gap is the "Temporal Dead Zone".
```

Q10. Event loop, call stack, microtasks vs macrotasks — predict the output.

```

console.log('1');
setTimeout(() => console.log('2'), 0);           // macrotask queue
Promise.resolve().then(() => console.log('3')); // microtask queue
console.log('4');

// Output: 1, 4, 3, 2
// Synchronous code runs first (1, 4). Then ALL microtasks (Promise .then) drain
// before the next macrotask (setTimeout) is even considered.

```

Q11. Prototypal inheritance — how does JS “class” syntax work under the hood?

```

class Animal {
  constructor(name) { this.name = name; }
  speak() { console.log(`${this.name} makes a sound`); }
}
class Dog extends Animal {
  speak() { console.log(`${this.name} barks`); }
}

// Under the hood: Dog.prototype.__proto__ === Animal.prototype
// 'class' is syntactic sugar over JS's existing prototype chain -- there's no
// separate "class" mechanism in the engine; method lookup walks the prototype chain.

```

Closures

Q12. Define a closure and show the classic “private counter” pattern.

A closure is a function that retains access to variables from its enclosing lexical scope even after that outer function has returned.

```
function createCounter() {
  let count = 0; // this variable is "closed over"
  return {
    increment: () => ++count,
    decrement: () => --count,
    getValue: () => count
  };
}
const counter = createCounter();
counter.increment(); counter.increment();
console.log(counter.getValue()); // 2
// 'count' is fully private -- no way to access it except through the returned methods.
```

Q13. Closures-in-a-loop bug — the classic interview question, with both fixes.

```
// BUG
function createFunctions() {
  var fns = [];
  for (var i = 0; i < 3; i++) {
    fns.push(function () { console.log(i); });
  }
  return fns;
}
createFunctions().forEach(fn => fn()); // 3, 3, 3 -- all closures share the one 'i'

// FIX 1: let (block-scoped, new binding per iteration)
for (let i = 0; i < 3; i++) { fns.push(() => console.log(i)); } // 0, 1, 2

// FIX 2: IIFE to create a new scope per iteration (the pre-ES6 way)
for (var i = 0; i < 3; i++) {
  (function (capturedI) { fns.push(() => console.log(capturedI)); })(i);
}
```

Q14. Memory leak risk with closures — how can they cause one?

A closure keeps its entire enclosing scope alive as long as the closure itself is reachable – including large objects you didn't mean to retain.

```
function setup() {
  const hugeData = new Array(1000000).fill('x'); // large object
  const smallHelper = () => console.log('done');
  return smallHelper; // hugeData is NOT used by smallHelper, but stays "in scope"
}
```

Practical takeaway: be deliberate about what large data structures live inside a closure that outlives its creator (e.g., an event handler registered once but referencing a big dataset).

Promises & Async/Await

Q15. Promise states and the classic “unhandled rejection” bug.

```
// BUG: no .catch(), rejection is silently swallowed / logged as unhandled
fetchUser(id).then(user => console.log(user.name));

// FIX
fetchUser(id)
  .then(user => console.log(user.name))
  .catch(err => console.error('Failed to fetch user:', err));
```

Q16. Promise.all vs allSettled vs race vs any — with a bug from picking the wrong one.

```
// BUG: Promise.all fails FAST -- if ANY promise rejects, you lose all other results
const results = await Promise.all([fetchA(), fetchB(), fetchC()]);
// if fetchB() rejects, you get NOTHING from fetchA/fetchC even though they succeeded

// FIX: allSettled always resolves, giving you status per-promise
const results = await Promise.allSettled([fetchA(), fetchB(), fetchC()]);
results.forEach(r => r.status === 'fulfilled' ? console.log(r.value) : console.error(r.reason));

// race: resolves/rejects as soon as the FIRST promise settles (useful for timeouts)
const result = await Promise.race([fetchData(), timeout(5000)]);

// any: resolves as soon as the FIRST one FULFILLS (ignores rejections unless all reject)
const first = await Promise.any([mirror1(), mirror2(), mirror3()]);
```

Q17. Sequential vs parallel awaits — a real performance bug.

```
// BUG: sequential when they don't depend on each other -- wastes time (3x latency)
async function loadDashboard() {
  const user = await fetchUser(); // ~200ms
  const orders = await fetchOrders(); // then another ~200ms
  const stats = await fetchStats(); // then another ~200ms
  return { user, orders, stats }; // total: ~600ms
}

// FIX: fire independent requests in parallel
async function loadDashboard() {
  const [user, orders, stats] = await Promise.all([
    fetchUser(), fetchOrders(), fetchStats()
  ]);
  return { user, orders, stats }; // total: ~200ms (bounded by the slowest one)
}
```

Q18. async function return value — what does it actually return?

```
async function getValue() { return 42; }
getValue(); // returns Promise<42>, NOT 42 directly
getValue().then(v => console.log(v)); // 42

// BUG: forgetting this and trying to use the value synchronously
const value = getValue();
console.log(value + 1); // "[object Promise]1" -- string concatenation bug, not 43
```

Q19. Error handling in async/await — the try/catch scope bug.

```

// BUG: doSomethingAsync() is not awaited -- if it rejects, this catch will NOT catch it
async function process() {
  try {
    doSomethingAsync();
  } catch (err) {
    console.error('caught:', err); // never runs for the unawaited rejection above
  }
}

// FIX: always await promises you want try/catch to actually cover
async function process() {
  try {
    await doSomethingAsync();
  } catch (err) {
    console.error('caught:', err);
  }
}

```

Q20. Retry logic with exponential backoff (realistic senior-level ask).

```

async function fetchWithRetry(fn, retries = 3, delayMs = 500) {
  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      return await fn();
    } catch (err) {
      if (attempt === retries) throw err; // exhausted retries, rethrow
      const backoff = delayMs * Math.pow(2, attempt - 1); // exponential
      await new Promise(resolve => setTimeout(resolve, backoff));
    }
  }
}

```

Azure

(mapped to the JD: App Services, Azure SQL, Storage, Functions, Key Vault)

Q21. Azure App Service — Deployment Slots. Slots let you deploy to a “staging” instance with its own hostname, warm it up, run smoke tests, then **swap** with production – a near-zero-downtime deployment (swap is a VIP swap, not a redeploy). Rolling back is just swapping back.

Q22. Azure Functions — trigger types and cold starts. Triggers: HTTP, Timer (cron-like), Queue (Service Bus/Storage Queue), Blob (on file upload), Event Grid. Consumption plan = pay-per-execution but has **cold starts** (first invocation after idle spins up a new instance, adds latency); Premium plan keeps instances warm for latency-sensitive workloads.

Q23. Azure Storage — Blob vs Table vs Queue. Blob = unstructured files (images, documents, backups), with hot/cool/archive tiers for cost. Table = NoSQL key-value, cheap, simple partition-key/row-key lookups (no complex queries/joins). Queue = simple message passing between services, decoupling producer from consumer (vs Service Bus, which adds topics/subscriptions, dead-lettering, sessions for more complex messaging).

Q24. Key Vault + Managed Identity — the full flow. App Service/AKS pod is assigned a Managed Identity (system- or user-assigned) -> Key Vault access policy (or RBAC role) grants that identity `get / list secret` permissions -> app code calls `builder.Configuration.AddAzureKeyVault(vaultUri, new DefaultAzureCredential())` -> secrets merge into `IConfiguration` at startup, no credential ever stored in code or config files.

Q25. Azure SQL — DTU vs vCore purchasing model. DTU: bundled compute+storage+IO in preset tiers, simpler for smaller/predictable workloads. vCore: independently scale compute and storage, choose your own hardware generation, better cost transparency for larger workloads, supports Azure Hybrid Benefit (bring your own SQL Server license for savings).

Q26. Azure Front Door — Classic vs Premium. Front Door is a global entry point (CDN + reverse proxy + WAF) routing users to the closest healthy backend. Premium adds tighter WAF rule management, Private Link support to backends (no public backend exposure), and managed rule sets updated by Microsoft.

Kubernetes (AKS)

Q27. ConfigMaps vs Secrets — why separate objects? Both inject config into Pods (as env vars or mounted files), but Secrets are base64-encoded (not encrypted at rest by default unless you enable encryption-at-rest / integrate Key Vault via the CSI driver) and access can be restricted via RBAC separately from ConfigMaps – keeping the security boundary distinct even though the mechanism looks similar.

Q28. Liveness vs Readiness probes — a bug from confusing them.

```
livenessProbe:
  httpGet: { path: /healthz, port: 8080 }
  periodSeconds: 10
readinessProbe:
  httpGet: { path: /ready, port: 8080 }
  periodSeconds: 5
```

Liveness failing -> Kubernetes **kills and restarts** the container (thinks the app is deadlocked/crashed). Readiness failing -> Kubernetes just **stops routing traffic** to that Pod, without restarting it. Bug: pointing both at the same slow endpoint (e.g., one checking DB connectivity) makes a slow DB look like a crashed app, causing repeated restarts. Fix: liveness checks “is the process alive” (cheap, no external calls); readiness checks “can it serve traffic right now” (can include dependency checks).

Q29. Horizontal Pod Autoscaler (HPA) — how does it decide to scale? Polls metrics (CPU/memory by default, or custom metrics via an adapter) against a target utilization you set (e.g., 70% CPU) and adjusts replica count within `minReplicas / maxReplicas` bounds. Requires resource `requests` to be set on the Pod spec – without a request value, HPA has no baseline to calculate percentage utilization against.

Q30. Rolling update vs recreate deployment strategy. Rolling update (default): gradually replaces old Pods with new ones, keeping some capacity available throughout – zero downtime if paired with readiness probes. Recreate: kills all old Pods first, then starts new ones – causes downtime, only used when the app can’t tolerate two versions running simultaneously (e.g., a schema-incompatible DB migration mid-rollout).

Docker

Q31. Layer caching — why instruction order matters for build speed.

```
# BAD: copying everything before restoring dependencies invalidates the cache
# on every single source code change, forcing a full package re-download every build
COPY . .
RUN dotnet restore
RUN dotnet build

# GOOD: copy only the project file first -- this layer only invalidates when
# dependencies actually change, not on every code edit
COPY *.csproj ./
RUN dotnet restore
COPY . .
RUN dotnet build
```

Q32. Why avoid running containers as root — and the fix.

Root inside a container, if a container-escape vulnerability exists, has a much larger blast radius on the host.

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0
RUN adduser --disabled-password --gecos '' appuser
USER appuser
COPY --chown=appuser:appuser . .
```

Q33. CMD vs ENTRYPOINT — the practical difference. `ENTRYPOINT` defines the fixed executable that always runs; `CMD` provides default *arguments* to it (overridable at `docker run`). Combining them is the common pattern: `ENTRYPOINT ["dotnet", "MyApp.dll"]` with `CMD ["--environment=Production"]` lets you override just the environment at runtime without redefining the whole entrypoint.

Q34. Docker networking basics — bridge vs host vs none. Bridge (default): container gets its own isolated network namespace and virtual NIC; ports must be explicitly published (`-p 8080:80`) to be reachable from the host. Host: container shares the host's network stack directly (no port mapping needed, but no isolation). None: no networking at all, for isolated batch jobs.